

A Large-Scale Analysis of AI-Assisted Pull Requests: Observations, Biases, and Limits of Behavioral Inference

Ahmed Mursal

School of Computing

Edinburgh Napier University

Edinburgh, Scotland

40646515@live.napier.ac.uk

Abstract—Through this analysis, the paper examines whether large-scale observational pull request datasets labeled with AI agent information can, in practice, support valid behavioral inference about AI coding agents. By tracing the full analytical trajectory—from initially compelling statistical signals to empirical validation and structural diagnosis—the study identifies the specific dataset properties that undermine inference, including extreme contribution inequality, automation and single-user dominance, attribution ambiguity, non-independence, measurement error in keyword-based detection, and widespread data loss. The work further evaluates the reliability of common metadata-driven proxies for testing behavior and demonstrates that, without manual validation, such proxies systematically overestimate test inclusion. While these limitations preclude defensible agent-level behavioral claims, the analysis shows that certain structural and distributional properties of the dataset remain empirically interpretable. Based on these findings, the paper distills methodological guidance for future research, emphasizing validation-first study design, explicit measurement error quantification, and dataset construction practices that are necessary for reliable empirical analysis of AI-assisted software development.

I. INTRODUCTION

AI-powered coding assistants are now embedded in everyday software development workflows. Tools such as GitHub Copilot, OpenAI Codex, Cursor, Devin, and Claude Code are used by millions of developers and are increasingly credited with accelerating development, improving productivity, and influencing software quality practices. As these tools become pervasive, understanding how they shape real-world development behavior—particularly testing practices—has become an important empirical question for both researchers and practitioners.

Recent software engineering research has responded to this need by leveraging large observational datasets derived from development platforms such as GitHub. These datasets appear especially attractive: they capture real production activity at unprecedented scale, promise high statistical power, and seemingly allow direct comparison of AI agent behavior “in the wild.” In particular, pull request (PR) datasets labeled with AI agent information have been used to make claims about differences in testing behavior, development style, and quality practices across tools.

This study began with exactly that motivation. Our original goal was to determine whether different AI coding agents exhibit systematically different testing behaviors. Using a large PR dataset with agent labels, we initially observed striking patterns: some agents appeared to include tests in nearly all PRs, while others rarely did so. Standard statistical analysis suggested large, statistically significant differences, seemingly supporting the narrative that AI agents differ meaningfully in how they support testing.

However, as the analysis progressed, these early findings raised a more fundamental question: what exactly do these apparent differences represent?

Closer inspection of the data revealed unexpected and troubling characteristics. Contribution activity was extremely concentrated, with a small number of users responsible for a disproportionate fraction of PRs. Some agent categories were dominated by single accounts producing thousands of PRs. A substantial portion of PRs were no longer accessible, preventing direct verification of their contents. Manual inspection further showed that keyword-based indicators of testing often failed to correspond to actual test files or meaningful test quality.

These observations forced a reassessment of the original research question. Rather than asking whether AI agents behave differently, this work reframes the problem: can observational PR datasets, as currently constructed, support valid behavioral inference about AI coding agents at all?

In response, this paper presents a structured empirical investigation that combines large-scale quantitative analysis with targeted manual validation. We document what the dataset clearly shows—such as adoption patterns and extreme contribution inequality—while also identifying where inference breaks down due to attribution ambiguity, measurement error, automation, and missing data. Rather than advancing unsupported behavioral claims, this study offers a methodological account of how seemingly compelling results emerge, why they are fragile, and how future research can avoid similar pitfalls.

By telling the full analytical story—from initial hypothesis, through failure, to methodological insight—this work

contributes both empirical observations and a cautionary framework for AI tool research based on large observational datasets.

A. Research Questions

This study investigates whether large-scale observational pull request datasets labeled with AI agent information can support valid behavioral inference about AI coding agents. Specifically, it examines (i) whether such datasets satisfy the assumptions required for agent-level behavioral comparison, (ii) which structural and measurement properties undermine inference, (iii) the reliability of metadata- and keyword-based proxies for testing behavior, and (iv) which empirical properties remain valid despite these limitations. Based on these findings, the study derives methodological guidance for future empirical research on AI-assisted software development.

II. DATASET ANALYSIS

A. Dataset Source

This study uses the AIDev dataset released as part of the MSR 2026 Mining Challenge. The dataset was collected by Li et al. and made publicly available to support research on AI-assisted software development. It aggregates GitHub pull request data annotated with AI agent attribution labels, representing one of the largest publicly available datasets of AI-assisted development activity.

The dataset was obtained directly from the official MSR 2026 challenge release and was used without modification to its original labels or structure, except where explicitly noted for analysis and validation purposes.

B. Dataset Scope and Composition

The full dataset contains:

- 932,791 pull requests
- 72,189 unique users
- Five AI agent labels:
 - OpenAI Codex
 - GitHub Copilot
 - Cursor
 - Devin
 - Claude Code
- Each pull request record includes:
 - Agent label
 - User identifier
 - Repository information
 - Pull request title and description
 - Timestamps and merge status
 - Links to the original GitHub PR (when available)

The dataset spans a large number of repositories, programming languages, and development contexts, providing broad coverage of real-world AI-assisted development activity.

TABLE I
DATASET SUMMARY

Metric	Value
Total PRs	932,791
Unique users	72,189
Gini coefficient	0.83
Inaccessible PRs in manual review	238
Agents	5

C. Intended Use and Initial Assumptions

The dataset is explicitly designed to support observational analysis of AI-assisted development practices. At face value, its scale and breadth appear well suited for behavioral comparison across agents, including analysis of testing practices via PR metadata.

However, as this study demonstrates, the suitability of the dataset for such inference depends critically on assumptions about attribution accuracy, data completeness, user independence, and measurement validity. These assumptions are examined empirically rather than taken for granted.

III. METHODOLOGY

IV. RESULTS

This section reports empirical findings derived from the dataset and from manual validation. Results are presented descriptively and diagnostically rather than as behavioral conclusions about AI agents. Where statistical measures are reported, they are used to characterize distributional properties and structural patterns, not to infer causality or agent intent.

A. Dataset-Level Structural Characteristics

Analysis of the full dataset (932,791 pull requests) reveals extreme structural imbalance in contribution activity.

Contribution inequality is severe. A small fraction of users account for a disproportionate share of pull requests, with the top 1% of contributors responsible for over 40% of all PRs. The computed Gini coefficient (≈ 0.83) indicates an exceptionally concentrated distribution, exceeding inequality levels commonly observed in socio-economic systems.

This concentration is not uniform across agents. Certain agent categories are dominated by one or a handful of users, producing thousands of PRs each. These patterns are directly observable from contribution counts and are independent of any testing-related analysis.

B. Accessibility and Data Loss

A manual review of 385 pull requests by one reviewer revealed that 238 were inaccessible (404 errors). Inaccessible PRs (404 errors) are especially prevalent among high-volume users and certain agent categories.

This loss of accessibility has two measurable consequences:

- **Verification limits:** Direct confirmation of test presence, language, or framework is impossible for these PRs.

- **Sampling distortion:** Manual validation cannot be performed on a fully random subset of the dataset, as accessibility is not uniformly distributed.

As a result, all findings based on manual review apply only to the accessible subset and cannot be assumed to generalize without qualification.

C. Manual Validation of Test Presence and Quality

A stratified manual review of 385 pull requests (77 per agent) was conducted to establish ground truth for test presence and quality. For accessible PRs, test inclusion and quality were categorized using a four-level rubric: Extensive, Comprehensive, Included, and Low.

The review confirms that testing practices are heterogeneous across pull requests. Even within the same agent category, PRs range from extensive, well-structured testing to minimal or superficial test inclusion. Only a minority of reviewed PRs achieved the highest testing quality level.

Importantly, manual inspection demonstrates that keyword-based detection is an unreliable proxy for actual testing behavior. False positives arise from template language, documentation, and discussion text, while false negatives occur when tests are present but not described using conventional keywords.

Given the per-agent sample size, these results should be interpreted as diagnostic evidence of heterogeneity and measurement error, rather than precise estimates of agent-specific testing rates.

D. Measurement Error in Automated Detection

Bidirectional validation—reviewing both keyword-detected and non-detected PRs—shows that automated test detection systematically overestimates test presence. The magnitude of this error varies by language, repository conventions, and PR structure.

This finding is robust across agents and does not depend on attribution correctness. It reflects a measurement limitation inherent to PR-level metadata rather than a property of any specific tool.

E. Debiasing Effects on Aggregate Patterns

Applying user-level filtering and structural corrections materially alters aggregate adoption statistics. After removing extreme high-volume contributors and likely automation accounts, relative agent shares shift, and distributional diversity increases as measured by Shannon entropy.

These changes demonstrate that raw PR counts are not stable indicators of adoption or behavior and that structural biases must be addressed before any comparative analysis is attempted.

F. Summary of Empirical Findings

In summary, the results show that:

- Contribution activity is extremely concentrated and unevenly distributed.
- A non-trivial portion of the dataset is no longer verifiable.

- Testing presence and quality vary widely across PRs.
- Keyword-based detection is systematically error-prone.
- Dataset structure materially distorts aggregate statistics.

Crucially, these results describe properties of the dataset and measurement process rather than behavioral characteristics of AI agents.

V. THREATS TO VALIDITY

This section outlines threats to validity following standard empirical software engineering practice. Rather than treating these as post hoc disclaimers, many of these threats were directly measured or motivated key methodological decisions in the study.

A. Internal Validity

Internal validity is primarily threatened by the inability to attribute observed artifacts to AI agents rather than to human developers or automated systems.

Agent labels indicate association, not authorship. A pull request attributed to an agent does not guarantee that the agent generated the code, the tests, or the final changes. Human modification, post-processing, and review decisions are inseparable from agent output in the available data.

Automation contamination further complicates interpretation. Some agent categories are dominated by a small number of high-volume users or system-like accounts, introducing non-human behavior that cannot be distinguished reliably from genuine development activity.

Finally, manual verification involves human judgment. Although a structured rubric was used, classification of testing quality remains subjective and may introduce reviewer bias.

B. Construct Validity

Construct validity is limited by how “testing behavior” is operationalized. This study measures the presence and apparent quality of tests in pull requests, which does not capture test effectiveness, coverage, correctness, or long-term maintenance value.

Keyword-based detection is an imperfect proxy for testing. Despite extensive language and framework coverage, automated detection exhibits both false positives and false negatives, particularly in multi-language repositories and template-driven projects. Manual verification mitigates but does not eliminate this limitation.

Additionally, pull requests are an imperfect unit of analysis. They aggregate multiple commits, may omit intermediate changes, and do not preserve the full development process, limiting their suitability for fine-grained behavioral inference.

C. External Validity

External validity is constrained by the dataset’s scope. The data is limited to GitHub and may not generalize to private repositories, enterprise environments, or other development platforms.

The dataset represents a specific temporal snapshot. AI tools evolve rapidly, and observed patterns may not hold for newer versions or future development practices.

Moreover, the presence of inaccessible PRs (404 errors) introduces uncertainty about representativeness. Accessible PRs may systematically differ from inaccessible ones, particularly for high-volume or short-lived users.

D. Conclusion Validity

Conclusion validity is threatened by structural dependence in the data. Pull requests are clustered by user, repository, and project, violating independence assumptions required by many statistical tests.

Although descriptive statistics and inequality metrics remain valid, inferential statistics based on independence assumptions cannot be interpreted as evidence of behavioral differences. Where such analyses were performed in early phases, they are explicitly reported as invalidated and are not used to support conclusions.

This study employs a multi-stage empirical methodology designed to evaluate measurement validity, structural bias, and inference limits in large-scale pull request datasets. The methodology prioritizes transparency, reproducibility, and empirical validation over assumption-driven analysis. All analytical steps were performed sequentially, with later stages introduced only after limitations were identified in earlier ones.

VI. IMPACT AND IMPLICATIONS

This work has impact at three levels: empirical research practice, dataset construction, and industrial interpretation of AI tool studies. Its contribution lies not in asserting new behavioral claims about AI agents, but in establishing what can and cannot be responsibly inferred from large observational pull request datasets.

A. Impact on Empirical Software Engineering Research

The primary impact of this study is methodological. It demonstrates how apparently strong, statistically significant results can emerge from observational datasets even when core assumptions required for behavioral inference are violated. By documenting the full analytical trajectory—from initial hypothesis, through invalid inference, to empirical correction—this work provides a concrete example of how large-scale software engineering studies can fail silently if validation is omitted.

This has direct relevance for the growing body of research that relies on mined development artifacts to study AI-assisted programming. The findings show that dataset scale alone does not ensure validity, and that independence, attribution, and measurement assumptions must be empirically verified rather than implicitly accepted. As a result, this work contributes a cautionary framework for interpreting future studies that compare AI tools using PR-level metadata.

B. Impact on Dataset Design and Curation

The analysis reveals structural properties of the dataset that materially affect downstream inference, including extreme contribution inequality, automation contamination, and loss of accessibility over time. These properties are not incidental;

they shape the conclusions that can be drawn and the types of questions the data can support.

By identifying these issues empirically, this work provides actionable guidance for future dataset construction. In particular, it highlights the importance of persistent identifiers, archival preservation of PR content, and explicit instrumentation of AI tool usage. Without such design choices, datasets risk becoming unsuitable for behavioral research despite their size and apparent richness.

C. Impact on Industrial and Practitioner Interpretation

From an industry perspective, this work cautions against drawing operational conclusions from aggregate PR statistics about AI tools. Apparent differences in testing behavior, productivity, or quality may reflect dataset artifacts, automation patterns, or user concentration rather than intrinsic tool characteristics.

This has implications for organizations selecting AI tools, designing development workflows, or benchmarking AI-assisted productivity. The results suggest that observational metrics should be interpreted as signals requiring validation, not as definitive evidence of tool superiority or behavioral effect.

D. Broader Research Implications

More broadly, this study argues for treating methodological failure as a valid and valuable research outcome. By explicitly reporting invalidated analyses, structural biases, and measurement error, the work contributes to a more transparent and reliable empirical research culture. This perspective aligns with best practices emerging across empirical sciences and is particularly important as AI tooling becomes more complex and pervasive.

VII. RELATED WORK AND METHODOLOGICAL FOUNDATIONS

This work draws on prior research across three intersecting areas: (1) “methodological foundations for empirical software engineering”, (2) “large-scale observational datasets derived from software repositories”, and (3) “studies of AI-assisted programming tools”. Together, this literature provides both the analytical grounding for the present study and the context necessary to interpret its findings.

A. Empirical Software Engineering Methodology and Validity

Foundational work in empirical software engineering has long emphasized that statistical analysis must be grounded in valid measurement, appropriate sampling, and explicit consideration of threats to validity. Wohlin et al. provide a comprehensive framework for experimental and observational research, including well-established taxonomies of internal, external, construct, and conclusion validity threats that inform the structure of this study’s validation analysis [1]. Similarly, Kitchenham et al. outline early but enduring guidelines for empirical software engineering, stressing that dataset size alone does not guarantee valid inference and that measurement validity and study design must precede statistical testing [2].

Sampling and manual validation practices in this study follow precedents established in prior large-scale empirical work. Naghashzadeh et al. employ statistically justified sample size estimation combined with manual labeling to study developer behavior in Q&A ecosystems [3]. Their use of confidence intervals, conservative proportion estimates, and human verification directly informs the sampling strategy and validation methodology adopted here, particularly in contexts where full population verification is infeasible.

B. Large-Scale Observational Datasets and Repository Mining

A substantial body of software engineering research relies on large observational datasets mined from platforms such as GitHub and Stack Overflow. While these datasets enable analysis at unprecedented scale, prior work has documented significant limitations related to metadata reliability, missing artifacts, and inference from incomplete records. Baltes et al., through the SOTorrent project, demonstrate how version-aware and content-level reconstruction is necessary to avoid misleading conclusions based solely on surface metadata [4]. Their work highlights the risks of treating repository metadata as a faithful proxy for developer intent or behavior.

Similarly, Asaduzzaman et al. show how dataset incompleteness and structural bias can distort conclusions when artifacts are missing, deleted, or unevenly represented [5]. These findings motivate the present study’s emphasis on accessibility checks, manual verification, and explicit treatment of unavailable pull requests as a first-order validity concern rather than a secondary limitation.

C. AI-Assisted Programming and Coding Agents

Recent research has begun to examine the role of “AI-powered coding assistants” in software development workflows. Vaithilingam et al. investigate the gap between developer expectations and lived experience when using AI programming assistants, illustrating how perceived tool behavior may differ from actual outcomes [6]. Their findings underscore why large-scale observational datasets are appealing for studying AI-assisted development, while also cautioning against uncritical interpretation of usage signals.

Broader discussions of AI code generation, such as Finnie-Ansley et al. frame AI coding tools as potentially transformative technologies whose impact extends beyond productivity to development practices and software quality [7]. At the model level, Chen et al. introduce Codex and establish its central role in AI-assisted code generation, providing context for its prominence in real-world datasets without making claims about downstream developer behavior [8].

Importantly, existing work in this area has largely relied on controlled experiments, surveys, or small-scale observational studies. By contrast, this study examines the methodological challenges that arise when attempting to infer agent-level behavior from large, heterogeneous observational datasets labeled with AI attribution metadata.

D. Measurement, Testing Practices, and Statistical Foundations

Software testing has long been recognized as a critical dimension of software quality, motivating its frequent use as an outcome variable in empirical studies. Prior work examining developer practices on platforms such as Stack Overflow (e.g., Calefato et al. [9]; Mi et al. [10]) demonstrates both the value and the limitations of textual and metadata-based proxies for quality-related behavior. These studies inform the cautious operationalization of test presence and quality in the present analysis.

From a statistical perspective, this work follows established guidance on effect size interpretation and categorical data analysis. Cohen’s framework emphasizes that statistical significance does not imply practical or scientific importance, particularly in large samples [11]. Agresti’s treatment of categorical data analysis further clarifies the assumptions underlying chi-square tests and related measures, reinforcing the need to assess independence and clustering before interpreting association statistics [12].

E. Positioning of the Present Work

Building on this literature, the present study contributes a methodological analysis of AI-assisted pull request datasets rather than advancing unqualified behavioral claims. By combining large-scale quantitative analysis with manual verification and explicit validity assessment, this work aligns with prior empirical software engineering standards while addressing emerging challenges unique to AI-labeled observational data. In doing so, it complements existing research on AI coding assistants by clarifying what such datasets can—and cannot—support in terms of behavioral inference.

VIII. CONCLUSION

This paper’s primary contribution is methodological: it demonstrates why certain behavioral questions about AI coding agents cannot currently be answered using observational pull request datasets, and provides empirical techniques for detecting and avoiding such invalid inference.

This study set out to examine whether large observational pull request datasets can support behavioral inference about AI coding agents, particularly with respect to testing practices. While initial analysis appeared to reveal strong and statistically significant differences across tools, deeper empirical investigation demonstrated that these patterns do not support valid behavioral conclusions.

REFERENCES

- [1] C. Wohlin, P. Runeson, M. H’öst, M. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in Software Engineering*. Springer, 2012.
- [2] B. Kitchenham, S. L. Pfleeger, L. Pickard, P. W. Jones, D. C. Hoaglin, K. E. Emam, and J. Rosenberg, “Preliminary guidelines for empirical research in software engineering,” *IEEE Transactions on Software Engineering*, vol. 28, no. 8, pp. 721–734, 2002.
- [3] A. Naghashzadeh et al., “Developer behavior in q&a ecosystems: A large-scale empirical study,” in *Proceedings of the 44th International Conference on Software Engineering*. ACM, 2022.

- [4] S. Baltes, L. Dumani, C. Treude, and S. Diehl, "Sotorrent: Reconstructing and analyzing the evolution of stack overflow posts," in *Proceedings of the 15th International Conference on Mining Software Repositories*. ACM, 2018, pp. 319–330.
- [5] M. Asaduzzaman, A. Mashiyat, C. K. Roy, and K. A. Schneider, "Answering questions about unanswered questions of stack overflow," in *Proceedings of the 10th Working Conference on Mining Software Repositories*. IEEE, 2013, pp. 97–100.
- [6] P. Vaithilingam *et al.*, "Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models," in *Proceedings of the 44th International Conference on Software Engineering*. ACM, 2022.
- [7] J. Finnie-Ansley *et al.*, "Robots for the masses: A survey of code generation tools powered by large language models," in *Proceedings of the 44th International Conference on Software Engineering*. ACM, 2022.
- [8] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.
- [9] F. Calefato, F. Lanubile, and N. Novielli, "Ask me anything: A study of stack overflow questions related to software testing," in *Proceedings of the 40th International Conference on Software Engineering*. ACM, 2018, pp. 886–896.
- [10] H. Mi *et al.*, "An empirical study of the characteristics of popular stack overflow questions," in *Proceedings of the 24th Asia-Pacific Software Engineering Conference*. IEEE, 2017, pp. 144–153.
- [11] J. Cohen, *Statistical Power Analysis for the Behavioral Sciences*. Routledge, 1988.
- [12] A. Agresti, *An Introduction to Categorical Data Analysis*. Wiley, 2018.